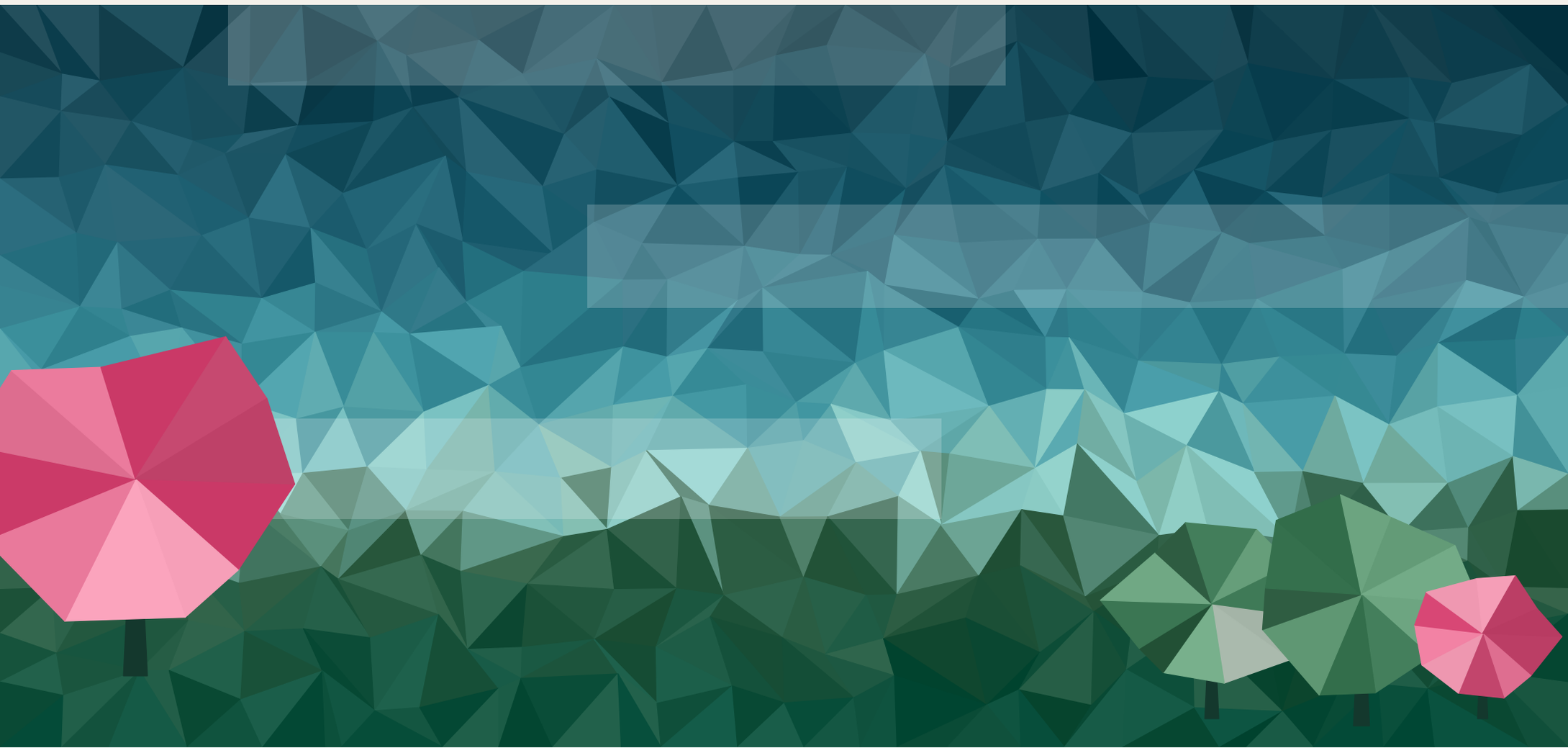
RELATÓRIO DE SRE · 2026

OBSERVABILIDADE, CUSTO E ROI

EcoCiente

EcoCiente IA (ia-eko) 26/10/2026 deepseek-flash · sem thinking US$ 1 = R$ 5,5884 código 583a8c9



CINCO PERGUNTAS DE SRE

Quanto custa, quão rápido responde, quanto erra, quanto retorna e quanto custa cada problema resolvido.

COMO LER ESTE RELATÓRIO

Cada número traz uma etiqueta de origem. Nenhum valor foi inventado: o que depende de tráfego real com a DeepSeek, de infraestrutura ou de benefício ainda não foi medido e aparece como tal.

MEDIDO saiu de uma execução do sistema, com o método descrito na página 11.

ESTIMADO calculado por fórmula a partir de premissas declaradas.

PREMISSA valor assumido, sem medição por trás.

A MEDIR ainda não existe número; o relatório indica como obtê-lo.



O RESUMO EM CINCO NÚMEROS

Cada cartão responde a um requisito do trabalho. As páginas seguintes mostram como chegamos a cada um.

CUSTO ESTIMADO

R$ 1,39

por semana · 100 usuários

Com 1.000 usuários: **R$ 13,90**. Caso esperado, só API; infraestrutura não informada.

ESTIMADO

TEMPO DE RESPOSTA

155 ms

p95 · 10 simultâneas

Com o LLM simulado: é o piso do sistema. Uma por vez: 20 ms. Com a DeepSeek: a medir.

MEDIDO A MEDIR

ÍNDICE DE ERROS

0 de 400

falhas 5xx no teste

Com o LLM simulado, que nunca falha. A taxa em produção depende da DeepSeek: a medir.

MEDIDO A MEDIR

CUSTO POR RESOLUÇÃO

R$ 0,0033

por resolução · caso esperado

Só API, com 85% de resolução assumida. Infraestrutura fora da conta.

ESTIMADO

CUSTO/RETORNO (ROI)

A DEFINIR

benefício por resolução

Fica positivo se cada resolução valer mais que **R$ 0,0033** (só API).

ESTIMADO PREMISSA

O QUE ESTE RELATÓRIO NÃO AFIRMA

Três limites que valem para todos os números.

- Que a resposta com a DeepSeek terá a latência medida aqui: o LLM foi simulado, então os tempos são do sistema sem o modelo.
- Que a taxa de resolução é de 100%: o juiz simulado aprova tudo. A taxa das contas, 85%, é uma premissa.
- Nada sobre infraestrutura nem benefício por resolução: ambos estão em zero no sistema, por não terem sido informados.

Custo de API por semana

ESTIMADO

100 usuários custam R$ 1,39 por semana e 1.000 usuários, R$ 13,90, no caso esperado.

Caso esperado · só custo de API

R$ 1,39 /semana

100 usuários · 500 solicitações

R$ 13,90 /semana

1.000 usuários · 5.000 solicitações

O custo de uma resposta vem dos tokens que ela consome e dos preços do **deepseek-flash sem thinking**, que mudam com o horário (pico ou fora do pico) e com o uso de cache.

Como ainda não há tráfego real com a DeepSeek, o tráfego no pico e o cache foram tratados como faixa, do melhor caso ao pior. Mil usuários custam exatamente 10 vezes mais em API.

O QUE ISSO SIGNIFICA

Nesta escala a API custa centavos por semana. O que vai pesar de verdade é a infraestrutura, que ainda não foi informada.

	100 usuários			1.000 usuários		
	Melhor	Caso esperado	Pior	Melhor	Caso esperado	Pior
Solicitações	500	500	500	5.000	5.000	5.000
Custo por solicitação	R$ 0,0015	R$ 0,0028	R$ 0,0066	R$ 0,0015	R$ 0,0028	R$ 0,0066
Custo de API	R$ 0,756	R$ 1,39	R$ 3,29	R$ 7,56	R$ 13,90	R$ 32,86
Resoluções	425	425	425	4.250	4.250	4.250

Melhor caso: fora do pico, 90% de cache. Caso esperado: 20,8% no pico, 50% de cache (tráfego uniforme na semana: 35 h de 168 h no pico). Pior caso: tudo no pico, sem cache.

Como o custo é calculado

Preço por token da DeepSeek, que muda com o horário e com o cache.

Preços usados (US$ por 1M de tokens)

	Pico	Fora do pico
Entrada, cache hit	0,006	0,003
Entrada, cache miss	0,30	0,15
Saída	1,20	0,60

Pico: 01h–04h e 06h–10h UTC (22h–01h e 03h–07h em Brasília), de segunda a sexta. A DeepSeek pode reajustar os preços.

Premissas PREMISSA

Solicitações por usuário por semana	5
Tokens de entrada por resposta	2.400
Tokens de saída por resposta	380
Respostas resolvidas	85%
Infraestrutura por semana	não informada

O QUE ISSO SIGNIFICA

O preço é previsível: depende dos tokens e de quanto do prompt cai no cache. O que ainda pode mexer no custo é o tamanho real das respostas, que será medido com a DeepSeek.

FÓRMULAS

```
preço(campo)          = fatia_pico × pico(campo) + (1 - fatia_pico) × fora_do_pico(campo)
custo da resposta US$ = [entrada × (cache × preço(hit) + (1 - cache) × preço(miss))
                         + saída × preço(saída)] ÷ 1.000.000
custo por solicitação = custo da resposta × cotação
custo de API semanal  = usuários × solicitações por usuário × custo por solicitação
```

Fonte dos preços: api-docs.deepseek.com/quick_start/pricing, consultada em 26/10/2026. Cotação: Frankfurter (api.frankfurter.dev), US$ 1 = R$ 5,5884 em 26/10/2026. As premissas são as mesmas do painel de controle e podem ser editadas lá.

Tempo total de resposta

MEDIDO

Sem o modelo, o sistema responde em 14 ms (mediana, uma requisição por vez).

p95 do tempo total · LLM simulado

20 ms

uma requisição por vez · 200 requisições

155 ms

10 simultâneas · 200 requisições

As medições vieram do sistema real rodando com o **LLM simulado** (`LLM_PROVIDER=mock`): o modelo responde na hora.

Elas mostram o custo de tempo do próprio sistema: guardrails, memória, roteamento, juiz e serialização. O tempo esperando a DeepSeek **não** está incluído A MEDIR.

O QUE ISSO SIGNIFICA

Estes tempos são um piso. O usuário só vai sentir a latência real quando o modelo da DeepSeek entrar na conta.

	Média	Mediana	p95	p99	Erros	Vazão
Uma por vez	15,0 ms	14,2 ms	20,1 ms	28,0 ms	0,00%	66,5 req/s
10 simultâneas	121,3 ms	118,8 ms	154,7 ms	166,0 ms	0,00%	81,6 req/s

Medido pelo cliente, com `scripts/benchmark_api.py`. Máquina de desenvolvimento (Windows, Python 3.14), sem rede externa, sem bancos, API e cliente no mesmo computador. Resultado de uma execução; não é capacidade de produção.

Latência entre agentes

MEDIDO

O especialista educacional concentra 96% do tempo das etapas do grafo.

O roteamento caiu sempre nele (400 de 400 decisões); as outras rotas não foram exercitadas.

Etapa	Chamadas	Média	p50	p95	Escala do p95 (0 a 39 ms)
Guardrail de entrada	400	< 0,1 ms	< 5 ms	< 5 ms	
Memória	400	< 0,1 ms	< 5 ms	< 5 ms	
Verificar compactação	400	< 0,1 ms	< 5 ms	< 5 ms	
Orquestrador	400	0,1 ms	< 5 ms	< 5 ms	
Educacional	400	12 ms	5,3 ms	39 ms	
Juiz de saída	400	0,1 ms	< 5 ms	< 5 ms	
Guardrail de saída	400	0,1 ms	< 5 ms	< 5 ms	
Entre as etapas	–	50 ms	–	–	fora das etapas do grafo
Total (servidor)	400	63 ms	50 ms	231 ms	

O QUE ISSO SIGNIFICA

Das 63 ms de média no servidor, só 12 ms estão nas etapas; o resto fica fora delas e as métricas atuais não o detalham.

O QUE MUDA COM A DEEPSEEK A MEDIR

No caminho padrão o grafo encadeia o orquestrador, o especialista e o juiz de saída, e cada um faz ao menos uma chamada ao LLM; se o juiz reprova, entram a correção e um novo juízo. FAQ via Qdrant e coletas não passam por juiz LLM.

O tempo total com a DeepSeek será o do sistema somado às chamadas ao modelo, o gargalo esperado.

COMO MEDIR

Rodar o benchmark com `LLM_PROVIDER=deepseek` e a chave no ambiente, e ler a latência por etapa e por chamada em `ecociente_agent_duration_seconds` e `ecociente_llm_duration_seconds`.

O projeto já alerta se o p95 do `/chat` passar de 15 s por 10 minutos (`alerts.yml`).

p50 e p95 são interpolações dos buckets do histograma (o menor vai até 5 ms): valem como ordem de grandeza. A média é exata.

Índice de erros MEDIDO

0 falhas em 400 requisições no teste com o LLM simulado.

Falhas 5xx · LLM simulado

0 de 400

0,00% das requisições

MEDIDO A MEDIR

Definição: índice de erros = requisições com falha de servidor (HTTP 5xx ou exceção) ÷ requisições totais.

Respostas 4xx (cliente), bloqueios do guardrail e recusas por limite de uso não entram: aparecem em contadores próprios.

O QUE ISSO SIGNIFICA

Zero erros prova pouco: o modelo simulado nunca falha, atrasa ou devolve saída inválida. A taxa real depende da DeepSeek.

Sinal	Resultado do teste	Métrica
Falhas 5xx	0 de 400 (0,00%)	ecociente_http_request_errors_total
Respostas reprovadas pelo juiz	0 de 400	ecociente_judge_decisions_total
Entradas barradas pelo guardrail	0	ecociente_guardrail_blocked_total
Recusadas por limite de uso	0	ecociente_quota_rejected_total

A carga foi pequena (400 requisições) e sem timeouts nem limites de uso da DeepSeek. Para acompanhar em produção: `sum(rate(ecociente_http_request_errors_total[5m])) / sum(rate(ecociente_http_requests_total[5m]))`.

Custo por resolução

ESTIMADO

Cada resolução custa R$ 0,0033 de API no caso esperado.

Por resolução · caso esperado · só API

R$ 0,0033

Faixa: R$ 0,0018 (melhor caso) a R$ 0,0077 (pior caso)

ESTIMADO PREMISSA

Definição: uma solicitação é *resolvida* quando a resposta final é aprovada pelo juiz de saída. Bloqueios do guardrail e respostas reprovadas após as correções não contam.

`custo por resolução = custo total ÷ resoluções`. As bloqueadas e reprovadas entram no custo, então quanto menos resolve, mais cada resolução custa.

O QUE ISSO SIGNIFICA

O custo por resolução é a métrica que une custo e qualidade: melhorar a taxa de aprovação do juiz reduz o custo de cada problema resolvido.

	100 usuários			1.000 usuários		
	Melhor	Caso esperado	Pior	Melhor	Caso esperado	Pior
Custo de API por resolução	R$ 0,0018	R$ 0,0033	R$ 0,0077	R$ 0,0018	R$ 0,0033	R$ 0,0077

Com 85% de resolução assumida. Não inclui infraestrutura, que ainda não foi informada: ao informá-la, soma-se ao custo total antes de dividir.

Custo/retorno (ROI)

ESTIMADO

Sem o benefício por resolução definido, não existe um ROI único.

Ponto de equilíbrio · caso esperado · só API

R$ 0,0033

benefício mínimo por resolução para o ROI deixar de ser negativo

ESTIMADO PREMISSA

Fórmula:

```
ROI = (benefício - custo total)
      ÷ custo total
benefício = resoluções
          × benefício por resolução
```

O benefício por resolução (por exemplo, um atendimento humano evitado) é uma decisão do grupo e ainda não foi informado.

O QUE ISSO SIGNIFICA

O grupo precisa escolher quanto vale uma resolução. A tabela mostra o que cada escolha resultaria.

Benefício por resolução (hipótese)	R$ 0,05	R$ 0,10	R$ 0,25	R$ 0,50	R$ 1,00
ROI com 100 usuários	+1.429%	+2.958%	+7.546%	+15.191%	+30.482%
ROI com 1.000 usuários	+1.429%	+2.958%	+7.546%	+15.191%	+30.482%

Os valores do cabeçalho são hipóteses de leitura, não estimativas do grupo. Caso esperado, só custo de API (infraestrutura em zero). Os percentuais são altos porque o custo de API por resolução é de frações de centavo; o ROI real cairá quando a infraestrutura for informada. É igual nos dois cenários porque a API escala linearmente; só diverge quando a infraestrutura de cada cenário for informada.

MÉTODO E LIMITAÇÕES

Tudo o que foi medido veio de uma máquina e de uma execução.

COMO AS MEDIÇÕES FORAM FEITAS

1. API do ia-eko iniciada localmente com `LLM_PROVIDER=mock`, `EMBEDDING_PROVIDER=mock` e `STORAGE_MODE=memory` (sem bancos, sem rede externa).
2. Duas execuções de `scripts/benchmark_api.py`, de 200 requisições cada, com concorrência 1 e 10; saídas em `relatorio/dados/`.
3. Leitura do `/metrics` e do `/api/v1/costs` ao final, com os mesmos histogramas que o painel usa. Percentis por interpolação linear nos buckets.
4. Custos pelas funções `simular` e `faixaPorSolicitacaoBrl` do painel (`dashboard/src/data/simular.ts`), testadas: relatório e painel mostram os mesmos números.

Para regerar: `node relatorio/gerar.mjs` a partir da raiz, com `npm install` feito em `dashboard/`.

LIMITAÇÕES

- **LLM simulado.** Latência, erros e resolução descrevem o sistema sem o modelo: um piso, não a experiência do usuário.
- **Rota única.** Só o especialista educacional foi exercitado; FAQ, analytics, coletas e grafo têm custos de tempo diferentes.
- **Uma máquina, uma execução.** Sem repetição nem carga sustentada.
- **Resolução e tokens são premissas.** O juiz simulado aprova tudo e o LLM simulado não reporta tokens.
- **Infraestrutura e benefício em zero.** Sem eles não há custo total completo nem ROI único.
- **Preços e cotação mudam.** Valem na data da consulta.

PARA FECHAR O QUE FALTA MEDIR

Quatro passos, nesta ordem. Os dois primeiros destravam quase todos os “a medir” deste relatório.

PRÓXIMOS PASSOS

1. Rodar o benchmark com a DeepSeek (`deepseek-flash`, sem thinking), em pico e fora do pico, com perguntas de todas as rotas; guardar tokens por resposta, latência por etapa e taxa de erro.
2. Informar a infraestrutura semanal para 100 e 1.000 usuários (`COST_INFRA_WEEKLY_BRL_100` e `_1000`).
3. Definir o benefício por resolução (`COST_BENEFIT_PER_RESOLUTION_BRL`) e medir as solicitações por usuário por semana com uso real.
4. Separar cache hit de cache miss no cálculo do backend: hoje o `cost.py` usa um único preço de entrada, o que superestima o custo.

REFERÊNCIAS

- DeepSeek, preços: api-docs.deepseek.com/quick_start/pricing
- Cotação: api.frankfurter.dev
- Repositório: `docs/SRE_E_CUSTOS.md`, `docs/PAINEL_DE_CONTROLE.md` e `observability/` (Prometheus e Grafana)